

Module 11: Spring Boot

Topic 10 (Part 5): Designing Standard Error Responses (ApiError / ErrorResponse DTO)

A **Standard Error Response** is a structured JSON object returned by a REST API whenever an error occurs. Instead of sending plain text or stack traces, the API returns a consistent format containing useful information about the error.

This is one of the **most important production-level concepts** in Spring Boot. Every modern enterprise REST API (Amazon, Google, Microsoft, Netflix, PayPal, etc.) returns structured error responses.

1. Introduction

Suppose a client requests:

GET /employees/101

If employee 101 exists:

HTTP/1.1 200 OK

```
{
  "id":101,
  "name":"Rahul"
}
```

Now suppose the employee does not exist.

Bad Response

HTTP/1.1 404 Not Found

Employee Not Found

The client only knows that something went wrong.

It does **not** know:

- When the error occurred
 - Which endpoint failed
 - What status code was returned
 - What type of error it was
-

Good Response

HTTP/1.1 404 Not Found

```
{
```

```
"timestamp":"2026-06-18T18:15:30",  
"status":404,  
"error":"Not Found",  
"message":"Employee with ID 101 not found.",  
"path":"/employees/101"  
}
```

This provides complete information for both developers and API consumers.

2. Why is a Standard Error Response Needed?

Imagine visiting a hospital.

Poor Report

Problem

Not useful.

Professional Report

Patient : Rahul

Disease : Fever

Temperature : 103°F

Doctor : Dr. Sharma

Date : 18 June 2026

This report is complete and easy to understand.

Similarly, REST APIs should return structured error information.

3. Problems Without Standard Error Responses

Different developers may return different formats.

Example 1

```
{
```

```
"message":"Employee Not Found"
}
```

Example 2

```
{
  "error":"Invalid ID"
}
```

Example 3

```
{
  "status":404
}
```

Example 4

Employee Missing

The frontend must handle many different formats, making development difficult.

A single standard solves this issue.

4. Typical Structure of an Error Response

Most enterprise APIs return:

ApiError

```
|
|— timestamp
|— status
|— error
|— message
└— path
```

Each field has a specific purpose.

5. Designing an ApiError DTO

Create a Java class.

```
public class ApiError {

    private LocalDateTime timestamp;
```

```
private int status;
```

```
private String error;
```

```
private String message;
```

```
private String path;
```

```
// Getters
```

```
// Setters
```

```
// Constructors
```

```
}
```

This class is called a **DTO (Data Transfer Object)** because it transfers data between the server and the client.

6. Understanding Each Field

A. timestamp

Stores when the error occurred.

Example:

```
"timestamp":"2026-06-18T18:15:30"
```

Useful for:

- Logging
- Debugging
- Auditing

B. status

Stores the HTTP status code.

Example:

"status":404

Possible values:

200

201

400

401

403

404

500

C. error

Stores the short error name.

Example:

"error":"Not Found"

Other examples:

Bad Request

Unauthorized

Conflict

Internal Server Error

D. message

Contains a detailed explanation.

Example:

```
"message": "Employee with ID 101 not found."
```

This is usually the most useful field for clients.

E. path

Stores the API endpoint.

Example:

```
"path": "/employees/101"
```

Useful when multiple APIs are running.

7. Complete Error Response

Example:

```
{  
  "timestamp": "2026-06-18T18:15:30",  
  "status": 404,  
  "error": "Not Found",  
  "message": "Employee with ID 101 not found.",  
  "path": "/employees/101"  
}
```

8. Integrating with @ControllerAdvice

Global Handler:

@ControllerAdvice

```
public class GlobalExceptionHandler {
```

```
    @ExceptionHandler(EmployeeNotFoundException.class)
```

```
    public ResponseEntity<ApiError> handle(  
  
        EmployeeNotFoundException ex,
```

```

        HttpServletRequest request){

    ApiError error = new ApiError();

    error.setTimestamp(LocalDateTime.now());

    error.setStatus(404);

    error.setError("Not Found");

    error.setMessage(ex.getMessage());

    error.setPath(request.getRequestURI());

    return ResponseEntity

        .status(HttpStatus.NOT_FOUND)

        .body(error);

}

}

```

Now every EmployeeNotFoundException returns the same format.

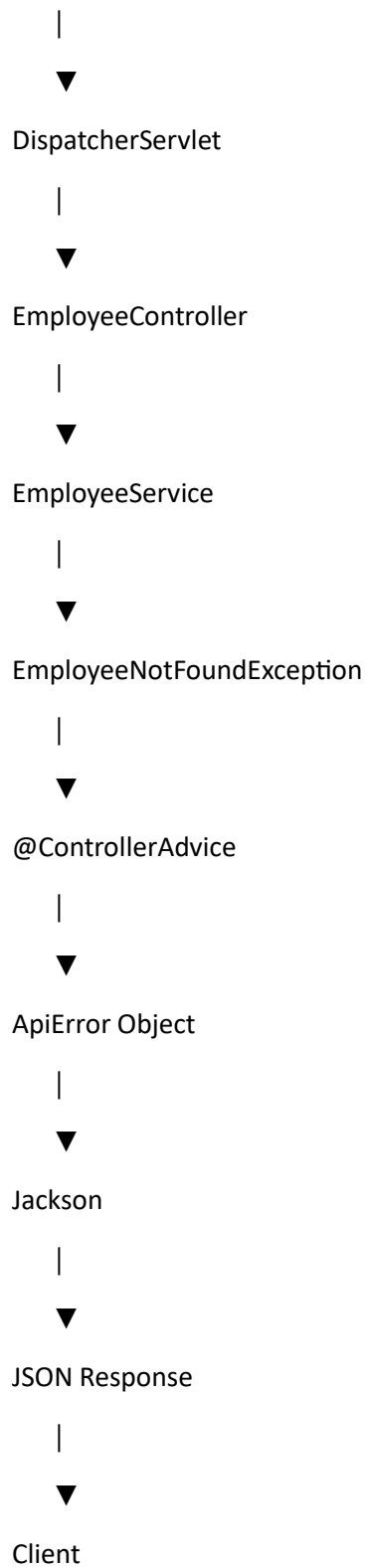
9. Internal Working

Suppose:

GET /employees/200

Execution:

Client



10. Validation Error Response

Suppose the client sends:

```
{
```

```
"name":"","  
"salary":-1000  
}
```

Validation fails.

A professional response might be:

```
{  
  "timestamp":"2026-06-18T18:20:00",  
  "status":400,  
  "error":"Bad Request",  
  "message":"Validation Failed",  
  "path":"/employees",  
  "errors":[  
    "Name must not be blank",  
    "Salary must be greater than 0"  
  ]  
}
```

Notice the additional errors field containing field-level validation messages.

11. Industry-Level ApiError

Large applications often include additional fields.

ApiError

```
|  
├— timestamp  
├— status  
├— error  
├— message  
├— path  
├— errorCode  
├— requestId  
├— errors  
└— traceId
```

Why?

- Easier debugging
 - Distributed microservices
 - Request tracking
 - Log correlation
 - Better frontend support
-

12. Complete Request Lifecycle

Client



HTTP Request



DispatcherServlet



Controller



Service



Repository



Exception



@ControllerAdvice



ApiError DTO



Jackson



JSON



HTTP Response

16. Advantages

- Consistent API responses.
- Easier frontend integration.
- Better debugging.
- Easier logging and monitoring.
- Professional API design.
- Improves maintainability.

17. Disadvantages

- Slightly more code than returning a plain string.
 - Requires careful design to avoid exposing sensitive information.
 - Must be kept consistent as the API evolves.
-

Module 11: Spring Boot

Topic 10 (Part 6): Validation Exception Handling (@Valid, MethodArgumentNotValidException, and Field Error Responses)

Validation Exception Handling is the process of detecting invalid user input, preventing it from reaching the business logic, and returning a structured, user-friendly error response to the client.

This is one of the **most important production-level concepts** in Spring Boot because **every REST API accepts input from users**, and that input must be validated before processing.

Examples:

- Registration Form
- Login API
- Employee Management System
- Banking Transactions
- E-commerce Checkout
- Payment Gateway
- Student Admission System

Without validation, invalid data can enter the database and cause application failures.

1. Introduction

Suppose an Employee API accepts the following request:

POST /employees

Content-Type: application/json

Request Body:

```
{  
  "name": "Rahul",  
  "salary": 50000  
}
```

This is valid.

The employee is saved successfully.

Now suppose the client sends:

```
{  
  "name": "",  
  "salary": -500  
}
```

Problems:

- Name is empty.
- Salary is negative.

Without validation, the application may store incorrect data.

With validation, Spring rejects the request and returns an error.

2. Why is Validation Exception Handling Needed?

Imagine a bank account opening form.

The customer submits:

Name :

Age : -20

Email : abc

Phone : 12

Without validation:

The bank stores incorrect data.

With validation:

The bank rejects the form and informs the customer:

Name cannot be blank

Age must be greater than 18

Invalid Email

Phone number is invalid

This is exactly how validation works in Spring Boot.

3. Historical Background

Before Bean Validation, developers manually wrote validation code.

Example:

```
if(employee.getName()==null ||
    employee.getName().isEmpty()){

    throw new RuntimeException(
        "Invalid Name");

}
```

Every field required manual checking.

Java later introduced **Bean Validation (JSR 303 / JSR 380)**.

Spring Boot integrates Bean Validation using annotations like:

@NotNull

@NotBlank

@Email

@Min

@Max

@Size

This greatly simplifies validation.

4. Bean Validation

Bean Validation uses annotations to declare validation rules.

Example:

```
public class Employee {  
  
    @NotBlank  
    private String name;  
  
    @Min(1)  
    private double salary;  
  
}
```

Instead of writing if statements, the annotations define the constraints.

5. Common Validation Annotations

Annotation	Purpose
@NotNull	Value must not be null
@NotBlank	String must not be null, empty, or whitespace
@NotEmpty	Collection/String must not be empty
@Size(min, max)	Length constraint
@Min(value)	Minimum numeric value
@Max(value)	Maximum numeric value
@Positive	Value must be greater than 0
@PositiveOrZero	Value must be ≥ 0
@Negative	Value must be less than 0
@Email	Valid email format
@Pattern	Matches a regular expression

Annotation	Purpose
@Past	Date must be in the past
@Future	Date must be in the future

6. Using @Valid

Spring performs validation when the controller parameter is annotated with @Valid.

Example:

```
@PostMapping("/employees")
public ResponseEntity<Employee> saveEmployee(

    @Valid
    @RequestBody Employee employee){

    return ResponseEntity.ok(employee);

}
```

What happens?

1. Request arrives.
 2. JSON is converted into an Employee object.
 3. Spring validates all annotated fields.
 4. If validation succeeds, the controller method executes.
 5. If validation fails, Spring throws a validation exception.
-

7. What is MethodArgumentNotValidException?

When validation fails, Spring throws:

MethodArgumentNotValidException

This exception contains:

- The invalid fields.
- Validation messages.
- Rejected values.

- Binding information.

It is automatically thrown before the controller method executes.

8. Internal Working

Suppose the client sends:

```
{  
  "name": "",  
  "salary": -1000  
}
```

Execution Flow:

Client

|



DispatcherServlet

|



Controller

|



@RequestBody

|



@Valid

|



Bean Validation

|



Validation Failed

|



MethodArgumentNotValidException

|



@ControllerAdvice

|



ApiError DTO

|



JSON Response

|



Client

Notice:

The controller method is **never executed** because validation fails first.

9. Handling MethodArgumentNotValidException

@ControllerAdvice

```
public class GlobalExceptionHandler {
```

```
    @ExceptionHandler(MethodArgumentNotValidException.class)
```

```
    public ResponseEntity<ApiError> handleValidation(
```

```
        MethodArgumentNotValidException ex){
```

```
        ApiError error = new ApiError();
```

```
        error.setStatus(400);
```

```
        error.setMessage("Validation Failed");
```

```

        return ResponseEntity
            .badRequest()
            .body(error);
    }

}

```

This returns a clean response instead of Spring's default validation output.

10. Extracting Field-Level Errors

The exception contains detailed information about each invalid field.

Example:

```
BindingResult result = ex.getBindingResult();
```

Retrieve all errors:

```
List<FieldError> fieldErrors =
    result.getFieldErrors();
```

Loop through them:

```
Map<String,String> errors = new HashMap<>();
```

```
for(FieldError error : fieldErrors){
```

```
    errors.put(
```

```
        error.getField(),
```

```
        error.getDefaultMessage()
```

```
    );
```

```
}
```

Result:

```
{  
  "name": "Name cannot be blank",  
  "salary": "Salary must be greater than zero"  
}
```

This format is very useful for frontend applications because each error is associated with a specific field.

11. Standard Validation Response

A professional validation response often looks like this:

```
{  
  "timestamp": "2026-06-18T19:30:00",  
  "status": 400,  
  "error": "Bad Request",  
  "message": "Validation Failed",  
  "path": "/employees",  
  "errors": {  
    "name": "Name cannot be blank",  
    "salary": "Salary must be greater than zero"  
  }  
}
```

12. Complete Request Lifecycle

Client

↓

POST /employees

↓

DispatcherServlet



JSON → Employee Object



@Valid



Bean Validation



Validation Failure



MethodArgumentNotValidException



@ControllerAdvice



ApiError DTO



Jackson



JSON



HTTP Response
